

D214 Data Analytics Graduate Capstone

NKM2 Data Analytics Report and Executive Summary (Task 2) Performance Assessment

Hillary Osei (Student ID #011039266)

Western Governors University, College of Information Technology

Program Mentor: Dan Estes

September 8, 2025

Table of Contents

A. Research Question.....	3
B. Data Collection.....	3
C. Data Extraction and Preparation.....	5
D. Analysis.....	24
E. Data Summary and Implications.....	28
F. Sources.....	32

A. Research Question

Cardiovascular disease (CVD) is the leading cause of death worldwide, responsible for millions of deaths annually and imposing significant burdens on healthcare systems. (CDC, 2024) Both preventable and genetic risk factors contribute to its diagnosis, making it a critical area of study for improving public health outcomes. Early detection and identification of risk factors can not only extend life expectancy but reduce healthcare costs and improve quality of life for at-risk populations.

Many potential factors of CVD, such as smoking status, alcohol use, physical activity, blood pressure, cholesterol, and glucose levels are commonly collected in healthcare/medical settings. Demographic factors like age and gender are also recognized as relevant to CVD outcomes. Investigating how these factors are associated with the presence of CVD can provide meaningful insights that support clinical decision-making and inform prevention strategies.

The research question guiding this analysis is “Do demographic, lifestyle, and clinical factors significantly impact the likelihood of cardiovascular disease?” To address this, the hypotheses are as follows:

- H_0 (null hypothesis): None of the predictor variables are significantly associated with cardiovascular disease.
- H_1 (alternative hypothesis): At least one variable is significantly associated with an increased likelihood of cardiovascular disease.

These hypotheses are tested using logistic regression, which allows for examining the statistical significance of each predictor on the risk of developing CVD.

B. Data Collection

The dataset used for this project is obtained from Kaggle’s “Cardiovascular disease dataset,” a compilation of two sources: the UCI Machine Learning repository’s Heart Disease dataset and a separate Kaggle “Heart Disease” dataset. (Welkins, n.d.) In total, it contains 68,205 records and 17 variables as follows:

Variable Name	Data Type
id	Continuous
age	Continuous
gender	Categorical

height	Continuous
weight	Continuous
ap_hi	Continuous
ap_lo	Continuous
cholesterol	Categorical
gluc	Categorical
alco	Categorical
active	Categorical
cardio (target variable)	Categorical
age_years	Continuous
bmi	Continuous
bp_category	Categorical
bp_category_encoded	Categorical

The dataset is publicly available, which makes it free to use without limitations and accessible for replication. An advantage of this dataset is its large sample size and balanced mix of categorical and continuous variables, which allows comprehensive analysis across different types of predictors. A limitation is that it is a secondary dataset compiled from multiple sources, which could raise the possibility of inconsistencies in measurement and data quality as well as uncertainty around how representative the data is of the broader population.

The data collection process is straightforward since the dataset is openly hosted and is downloaded directly from Kaggle without issue. A challenge encountered during the process was verifying the origins of the dataset and ensuring that its licensing permitted unrestricted use. This was addressed by reviewing the primary and secondary datasets' documentation, which confirmed it is publicly available licensed under Creative Commons Attribution 4.0 International or designated as public domain, allowing it to be used freely for analysis purposes.

C. Data Extraction and Preparation

Python is the primary environment for handling the dataset for handling the dataset because of its versatility and wide range of libraries. Pandas and NumPy are used for data manipulation, transformation, and handling numerical arrays. Matplotlib and Seaborn are used for creating visualizations exploring distributions and relationships between variables. Statsmodels is used to run the logistic regression model calculating p-values and odds ratios for interpreting the predictor's statistical significance. Scikit-learn is used for splitting the dataset into training and test sets, fitting logistic regression models, and generating evaluation metrics such as confusion matrices and accuracy scores. The advantage of using Python is that it enables the entire workflow (from data extraction to data cleaning, preparation, visualization, and statistical modeling) to occur within one reproducible environment. On the other hand, a disadvantage is that open-source data processing can be sensitive to errors in the code and misinterpretation of variables, which places the responsibility on the analyst to check each step carefully.

The data extraction process begins by downloading the “Cardiovascular Disease” dataset from Kaggle and loading it into Python as a CSV file using `pd.read_csv(“cardio_data_processed.csv”)`.

A preliminary inspection on the dataset is performed using `df.info()` and `df.head()` to review data structure and preview the first five rows.

```
[3]: # Get summary of dataframe
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 68205 entries, 0 to 68204
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    68205 non-null  int64
1   age                   68205 non-null  int64
2   gender                68205 non-null  int64
3   height                68205 non-null  int64
4   weight                68205 non-null  float64
5   ap_hi                 68205 non-null  int64
6   ap_lo                 68205 non-null  int64
7   cholesterol           68205 non-null  int64
8   gluc                  68205 non-null  int64
9   smoke                 68205 non-null  int64
10  alco                  68205 non-null  int64
11  active                68205 non-null  int64
12  cardio                68205 non-null  int64
13  age_years              68205 non-null  int64
14  bmi                   68205 non-null  float64
15  bp_category            68205 non-null  object
16  bp_category_encoded    68205 non-null  object
dtypes: float64(2), int64(13), object(2)
memory usage: 8.8+ MB
```

```
[4]: # Get preview of values in dataset
df.head()
```

```
[4]:
```

	id	age	gender	height	weight	ap_hi	ap_lo	cholesterol	gluc	smoke	alco	active	cardio	age_years	bmi	bp_category	bp_category_encoded
0	0	18393	2	168	62.0	110	80	1	1	0	0	1	0	50	21.967120	Hypertension Stage 1	Hypertension Stage 1
1	1	20228	1	156	85.0	140	90	3	1	0	0	1	1	55	34.927679	Hypertension Stage 2	Hypertension Stage 2
2	2	18857	1	165	64.0	130	70	3	1	0	0	0	1	51	23.507805	Hypertension Stage 1	Hypertension Stage 1
3	3	17623	2	169	82.0	150	100	1	1	0	0	1	1	48	28.710479	Hypertension Stage 2	Hypertension Stage 2
4	4	17474	1	156	56.0	100	60	1	1	0	0	0	0	47	23.011177	Normal	Normal

Columns irrelevant to the research question, including “id,” “age,” “height,” “weight,” “ap_hi,” “ap_lo,” and “bp_category_encoded” are dropped. The variable “id” is dropped because it only served as identifiers and had no predictive value. The variable “age” (represented in days) is dropped because a more standardized version “age_years” was available, which was later renamed to “age” for simplicity and clarity. Raw “height” and “weight” variables are removed because the variable “bmi,” already included, provides a standardized measure of body composition and is less prone to multicollinearity when used alongside height and weight. The systolic (“ap_hi”) and diastolic (“ap_lo”) blood pressure variables are excluded after blood pressure categories had already been encoded into an ordinal variable (“bp_category”), ensuring the data is not redundant. The variable “bp_category_encoded” is also dropped because the ordered “bp_category” variable already exists and is a duplicate of “bp_category_encoded.”

```
[5]: # Drop irrelevant columns
df = df.drop(columns = ['id', 'age', 'height', 'weight', 'bp_category_encoded', 'ap_hi', 'ap_lo'])
```

```
[6]: # Rename age_years
df = df.rename(columns = {'age_years': 'age'})
```

Missing values are detected using `df.isnull().sum()`, with results confirming that none are present in the dataset.

```
[8]: # Check for missing values  
df.isnull().sum()
```

```
[8]: gender          0  
     cholesterol    0  
     gluc           0  
     smoke          0  
     alco           0  
     active         0  
     cardio         0  
     age            0  
     bmi            0  
     bp_category    0  
     dtype: int64
```

Duplicate entries are also detected. Using `df.duplicated().sum()`, it is discovered that 5,652 records are exact duplicates.

```
[9]: # Check for sum of duplicates  
df.duplicated().sum()
```

```
[9]: 5652
```

Upon close inspection of the first 20 duplicate cases, it is revealed that several patients share identical demographic, clinical, and lifestyle factors, including age, cholesterol and glucose levels, smoking and alcohol use, activity status, BMI, and blood pressure category.

```
[10]: # Inspect sample of duplicates
      dupes = df[df.duplicated(keep=False)]
      dupes.sort_values(list(df.columns)).head(20) # look at first 20
```

	gender	cholesterol	gluc	smoke	alco	active	cardio	age	bmi	bp_category
4650	1	1	1	0	0	0	0	39	19.814053	Normal
28405	1	1	1	0	0	0	0	39	19.814053	Normal
29071	1	1	1	0	0	0	0	39	22.773186	Hypertension Stage 1
68054	1	1	1	0	0	0	0	39	22.773186	Hypertension Stage 1
5289	1	1	1	0	0	0	0	39	23.875115	Hypertension Stage 1
24557	1	1	1	0	0	0	0	39	23.875115	Hypertension Stage 1
50952	1	1	1	0	0	0	0	39	25.529645	Normal
54371	1	1	1	0	0	0	0	39	25.529645	Normal
17401	1	1	1	0	0	0	0	39	27.343750	Hypertension Stage 1
21238	1	1	1	0	0	0	0	39	27.343750	Hypertension Stage 1
43986	1	1	1	0	0	0	0	40	21.778435	Normal
67540	1	1	1	0	0	0	0	40	21.778435	Normal
39988	1	1	1	0	0	0	0	40	22.773186	Hypertension Stage 1
62374	1	1	1	0	0	0	0	40	22.773186	Hypertension Stage 1
8857	1	1	1	0	0	0	0	40	23.529412	Hypertension Stage 1
23261	1	1	1	0	0	0	0	40	23.529412	Hypertension Stage 1
40287	1	1	1	0	0	0	0	40	23.875115	Hypertension Stage 1
45043	1	1	1	0	0	0	0	40	23.875115	Hypertension Stage 1
51610	1	1	1	0	0	0	0	40	23.875115	Hypertension Stage 1
52816	1	1	1	0	0	0	0	40	23.875115	Hypertension Stage 1

Since logistic regression assumes independent observations (Bobbitt, 2020), retaining duplicates could skew the model results, specifically coefficients and p-values. Therefore, duplicates are dropped using `df.drop_duplicates()`, reducing the dataset size from 68,205 to 62,553 records. Doing this ensures that each observation contributes equally to the analysis.

```
[11]: # Check dataframe shape before dropping duplicates
      print("Before dropping:", df.shape)
      # Drop duplicates
      df = df.drop_duplicates()
      # Check dataframe shape after dropping duplicates
      print("After dropping:", df.shape)
```

Before dropping: (68205, 10)
After dropping: (62553, 10)

Frequency counts, using `value_counts()` are used to get categorical distributions. For the “gender” variable, it is determined that the majority of patients are female (39,957) compared to male (22,596).

```
[12]: # Get unique value count for "gender"
      df["gender"].value_counts() # 1 for female, 2 for male

[12]: gender
      1    39957
      2    22596
      Name: count, dtype: int64
```

Cholesterol levels show that 45,766 patients have normal levels, 9,112 have above normal levels, and 7,675 have well above normal levels.

```
[13]: # Get unique value count for "cholesterol"
      df["cholesterol"].value_counts() # 1 for normal, 2 for above normal, 3 for well above normal

[13]: cholesterol
      1    45766
      2     9112
      3     7675
      Name: count, dtype: int64
```

Glucose levels show a similar trend, with most patients (52,470) falling within the normal category.

```
[14]: # Get unique value count for "gluc"
      df["gluc"].value_counts() # 1 for normal, 2 for above normal, 3 for well above normal

[14]: gluc
      1    52470
      3     5106
      2     4977
      Name: count, dtype: int64
```

Lifestyle variables reveal that 56,611 patients are non-smokers compared to 5,942 smokers, 58,938 patients do not consume alcohol versus 3,615 patients, and 49,633 reported being physically active compared to 12,920.

```
[15]: # Get unique value count for "smoke"
      df["smoke"].value_counts() # 0 for non-smoker, 1 for smoker

[15]: smoke
      0    56611
      1     5942
      Name: count, dtype: int64

•[16]: # Get unique value count for "alco"
      df["alco"].value_counts() # 0 for doesn't consume alcohol, 1 for consumes alcohol

[16]: alco
      0    58938
      1     3615
      Name: count, dtype: int64

[17]: # Get unique value count for "active"
      df["active"].value_counts() # 0 for not physically active, 1 for physically active

[17]: active
      1    49633
      0    12920
      Name: count, dtype: int64
```

Blood pressure categories show that hypertension stage 1 is the most common among patients at 35,099, followed by hypertension stage 2 (15,441), normal (8,968), and elevated (3,045).

```
[18]: # Get unique value count for "bp_category"
      df["bp_category"].value_counts()

[18]: bp_category
      Hypertension Stage 1    35099
      Hypertension Stage 2    15441
      Normal                  8968
      Elevated                 3045
      Name: count, dtype: int64
```

Categorical variables are then encoded to make them suitable for logistic regression. The “gender” variable, originally coded as 1 for female and 3 for male, is recorded into a binary variable named “gender_male” where male = 1 and female = 0. This data transformation helps interpret the effect of being male on cardiovascular disease compared to female patients.

The “cholesterol” and “gluc” variables, which were originally ordinal with values "Normal," "Above Normal," and "Well Above Normal" are transformed into dummy variables using `pd.get_dummies()`. The “normal” category is dropped to serve as the baseline to prevent multicollinearity, which occurs when predictors are highly correlated (Bobbitt, 2021). The resulting dummy variables are renamed to reflect their meaning more clearly, such as

“cholesterol_above_normal” and “cholesterol_well_above_normal.” The same process is applied to glucose levels, producing “glucose_above_normal” and “glucose_well_above_normal.”

The “bp_category” variable is converted to an ordered categorical type with defined levels: “Normal,” “Elevated,” “Hypertension Stage 1,” and “Hypertension Stage 2.” After assigning the order, the variable is one-hot encoded into dummy variables, with “Normal” being dropped. All of these dummy variables are appended to the dataset using `pd.concat()`, and the original “cholesterol,” “gluc,” and “bp_category” variables are dropped.

```
[19]: # Convert values in gender to 0/1 (male = 1, female = 0); rename gender_name
df['gender_male'] = df['gender'].map({1: 0, 2: 1})
df.drop(columns=['gender'], inplace=True)

# Get dummy variables for 'cholesterol' and 'gluc,' drop first category to prevent multicollinearity
df = pd.get_dummies(df, columns=['cholesterol', 'gluc'], drop_first=True)

# Rename columns for clarity
df.rename(columns={
    'cholesterol_2': 'cholesterol_above_normal',
    'cholesterol_3': 'cholesterol_well_above_normal',
    'gluc_2': 'glucose_above_normal',
    'gluc_3': 'glucose_well_above_normal'
}, inplace=True)

# Convert bp_category into category data type
df['bp_category'] = df['bp_category'].astype('category')
# Define order for categories
order = ["Normal", "Elevated", "Hypertension Stage 1", "Hypertension Stage 2"]
df['bp_category'] = pd.Categorical(df['bp_category'], categories=order, ordered=True)

# Get dummy variables for bp_category
bp_dummies = pd.get_dummies(df['bp_category'], prefix='bp', dtype=int, drop_first=True)
# Set column names to lowercase, replace blank spaces with underscores
bp_dummies.columns = (bp_dummies.columns
    .str.lower()
    .str.replace(' ', '_'))

# Add bp_dummies to df
df = pd.concat([df, bp_dummies], axis=1)

# Drop bp_category
df.drop(columns=['bp_category'], inplace=True)
```

The dummy variables are then converted to binary values (0/1), using the following code:

```
[20]: # Convert dummy variables into binary value (0/1)
true_false = {True: 1, False: 0}
df["cholesterol_above_normal"] = df["cholesterol_above_normal"].replace(true_false).infer_objects(copy=False)
df["cholesterol_well_above_normal"] = df["cholesterol_well_above_normal"].replace(true_false).infer_objects(copy=False)
df["glucose_above_normal"] = df["glucose_above_normal"].replace(true_false).infer_objects(copy=False)
df["glucose_well_above_normal"] = df["glucose_well_above_normal"].replace(true_false).infer_objects(copy=False)
```

Continuous variables “age” and “bmi” are explored with descriptive statistics using `df.describe()`. The variable “age” spans from 29 to 64 years, with a mean of 52.83. The interquartile range is relatively narrow (from 48 to 58 years), and both the median (53 years) and mean indicate that the data is centered around middle-aged patients. This suggests that the dataset largely represents older adults, which aligns with populations at higher risk for cardiovascular disease.

For “bmi,” there is a wider spread. The mean BMI is 27.7, which is slightly above the clinical cutoff for overweight (≥ 25) (CDC, 2024), suggesting that overweight and obesity are prevalent in the patient population. The IQR (23.88 to 30.48) places the middle 50% of patients in the upper-normal to obese range. The median (26.58) is slightly lower than the mean, indicating right-skewness, which is also supported by the extreme maximum value of 298.67. The maximum and minimum BMI of 3.47 is implausible, confirming the presence of extreme outliers and possible data entry errors.

```
[23]: # Get summary statistics (pre-outlier handling)
      df[['age', 'bmi']].describe()
```

```
[23]:
```

	age	bmi
count	62553.000000	62553.000000
mean	52.826547	27.698470
std	6.826683	6.195161
min	29.000000	3.471784
25%	48.000000	23.875433
50%	53.000000	26.575891
75%	58.000000	30.477219
max	64.000000	298.666667

Outliers in “bmi” are handled using winsorization, a technique that caps extreme values at defined thresholds without dropping observations (Thevapalan, 2024). BMI values below the 1st percentile are replaced with the 1st percentile value, and values above the 99th percentile are replaced with the 99th percentile value. This approach preserves the overall dataset size while eliminating the influence of extreme values that could negatively impact the model’s accuracy.

```
[24]: # Handle outliers in bmi with winsorization
      lower, upper = df['bmi'].quantile([0.01, 0.99])
      df['bmi'] = df['bmi'].clip(lower, upper)
```

Summary statistics are recalculated solely for “bmi” after applying winsorization to adjust extreme values. The revised range now spans from 18.51 to 44.58, falling within plausible limits, compared to the extreme range of 3.47 to 298.66 before. The mean BMI decreased slightly from 27.70 to 27.61, while the standard deviation narrowed from 6.20 to 5.16, indicating reduced variability. The interquartile range (23.88 to 30.48) still places the middle 50% of patients in the upper-normal to obese range.

```
[25]: # Get summary statistics (post-outlier handling)
      df['bmi'].describe()
```

```
[25]: count    62553.000000
      mean      27.607666
      std       5.164575
      min      18.507766
      25%      23.875433
      50%      26.575891
      75%      30.477219
      max      44.581619
      Name: bmi, dtype: float64
```

Exploratory analysis through univariate and bivariate visualizations provides deeper insight into how demographic, lifestyle, and clinical variables relate to CVD. The distribution for “age” is left-skewed, with most patients falling between ages 30 and 65. There is one outlier present below the age of 30 but it is retained for variability. Boxplots comparing age by cardiovascular disease status reveal that patients diagnosed with CVD tend to be older, between 40 and 65 years old, while patients without the disease are somewhat younger, falling between 30 and 60 years. This suggests that age is a strong determinant of disease risk.

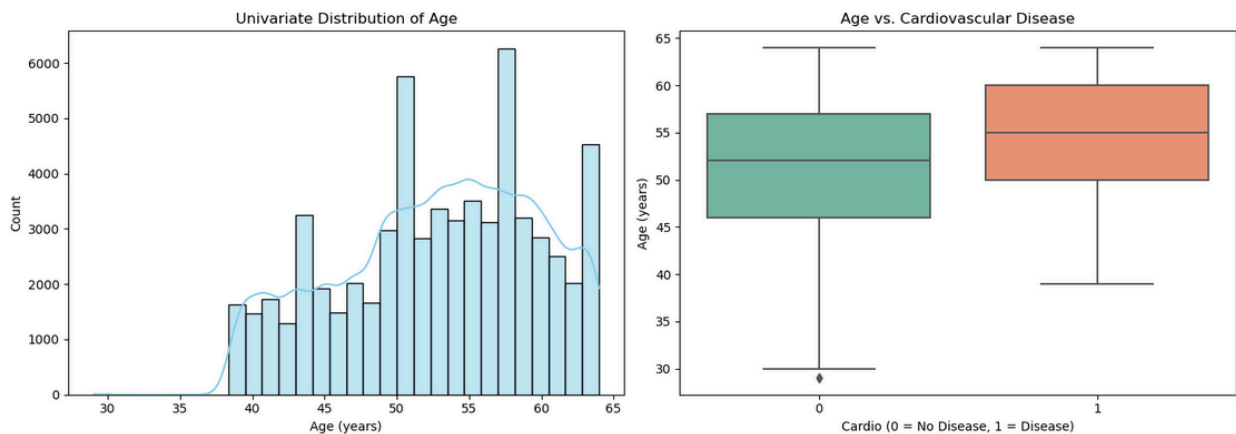
```
[26]: # Visualizations for age

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Left: Univariate (age) ---
sns.histplot(df['age'], bins=30, kde=True, ax=axes[0], color='skyblue')
axes[0].set_title("Univariate Distribution of Age")
axes[0].set_xlabel("Age (years)")
axes[0].set_ylabel("Count")

# --- Right: Bivariate (Age vs cardio) ---
sns.boxplot(x='cardio', y='age', data=df, ax=axes[1], palette="Set2")
axes[1].set_title("Age vs. Cardiovascular Disease")
axes[1].set_xlabel("Cardio (0 = No Disease, 1 = Disease)")
axes[1].set_ylabel("Age (years)")

plt.tight_layout()
plt.show()
```



The new distribution for “bmi” is right-skewed, with most patients clustering between 20 and 30 but a tail extending toward the higher values. The boxplots comparing BMI against CVD status show that patients with CVD tend to have higher BMI values than those without the disease, consistent with weight being a known risk factor. Outliers remain in both groups, however they do fall within realistic ranges after winsorization.

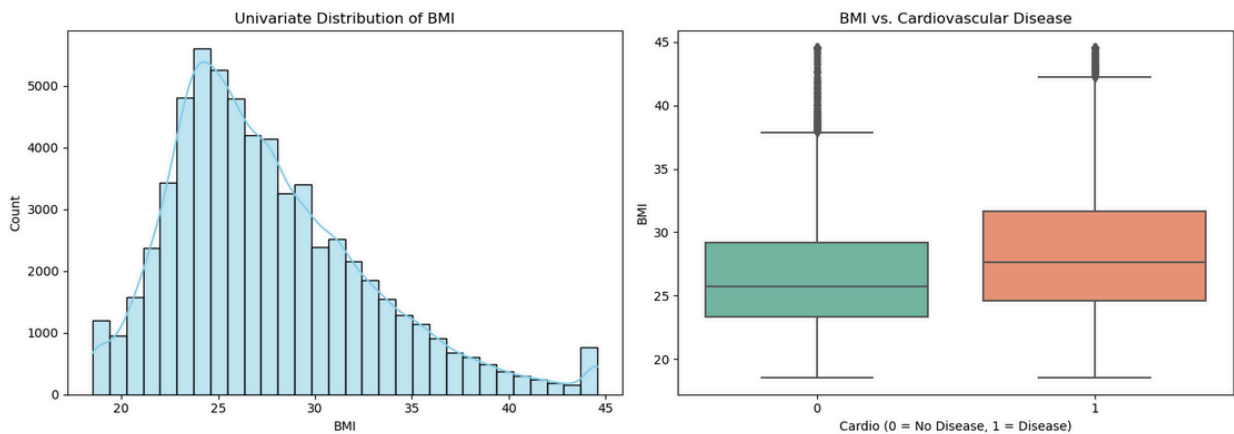
```
[27]: # Visualizations for bmi

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Left: Univariate BMI ---
sns.histplot(df['bmi'], bins=30, kde=True, ax=axes[0], color='skyblue')
axes[0].set_title("Univariate Distribution of BMI")
axes[0].set_xlabel("BMI")
axes[0].set_ylabel("Count")

# --- Right: Bivariate (BMI vs cardio) ---
sns.boxplot(x='cardio', y='bmi', data=df, ax=axes[1], palette="Set2")
axes[1].set_title("BMI vs. Cardiovascular Disease")
axes[1].set_xlabel("Cardio (0 = No Disease, 1 = Disease)")
axes[1].set_ylabel("BMI")

plt.tight_layout()
plt.show()
```



In the “smoke” variable, univariate plots show that the majority of patients are non-smokers. However, bivariate analysis reveals an unexpected pattern: among non-smokers, more patients have CVD, while among smokers, more patients do not. This may suggest that age, physical activity, or clinical conditions may influence disease prevalence more strongly than just smoking alone in the dataset.

```
[28]: # Visualizations for smoke

# Make readable labels
df['smoke_label'] = df['smoke'].map({0: 'Non-Smoker', 1: 'Smoker'})
df['cardio_label'] = df['cardio'].map({0: 'No Disease', 1: 'Disease'})

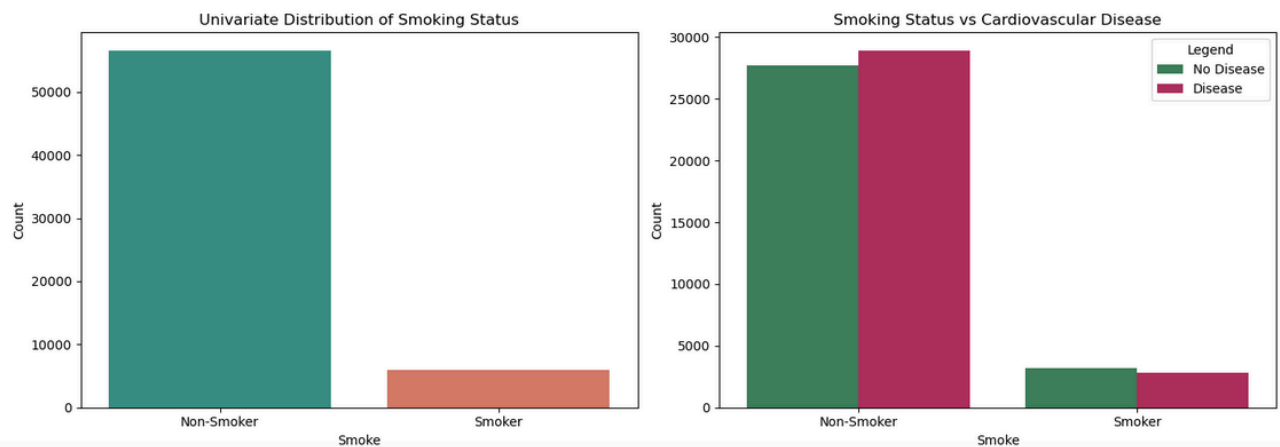
# Color palette
smoke_palette = ['#2A9D8F', '#E76F51'] # Non-Smoker, Smoker
cardio_palette = {'No Disease': '#2E8B57', 'Disease': '#C2185B'}

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Left: Univariate Smoker ---
sns.countplot(x='smoke_label', data=df, ax=axes[0], palette=smoke_palette)
axes[0].set_title('Univariate Distribution of Smoking Status')
axes[0].set_xlabel('Smoke')
axes[0].set_ylabel('Count')

# --- Right: Bivariate (Smoker vs cardio) ---
sns.countplot(x='smoke_label', hue='cardio_label', data=df,
              palette=cardio_palette, hue_order=['No Disease', 'Disease'], ax=axes[1])
axes[1].set_title('Smoking Status vs Cardiovascular Disease')
axes[1].set_xlabel('Smoke')
axes[1].set_ylabel('Count')
axes[1].legend(title='Legend')

plt.tight_layout()
plt.show()
```



For the “alco” variable, most patients do not drink according to univariate analysis, but bivariate proportions show that patients who consumed alcohol are less likely to have CVD compared to those who abstained. This counterintuitive finding suggests that alcohol use might not directly lower the risk of CVD, but could be linked to other factors, such as differences in diet, lifestyle, or socio-economic status which are influencing the results.

```
[29]: # Visualization for alco

# Make readable labels (avoid overwriting if they already exist)
tmp = df.assign(
    alco_label = df['alco'].map({0: 'Does Not Consume Alcohol', 1: 'Consumes Alcohol'}),
    cardio_label = df['cardio'].map({0: 'No Disease', 1: 'Disease'})
)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

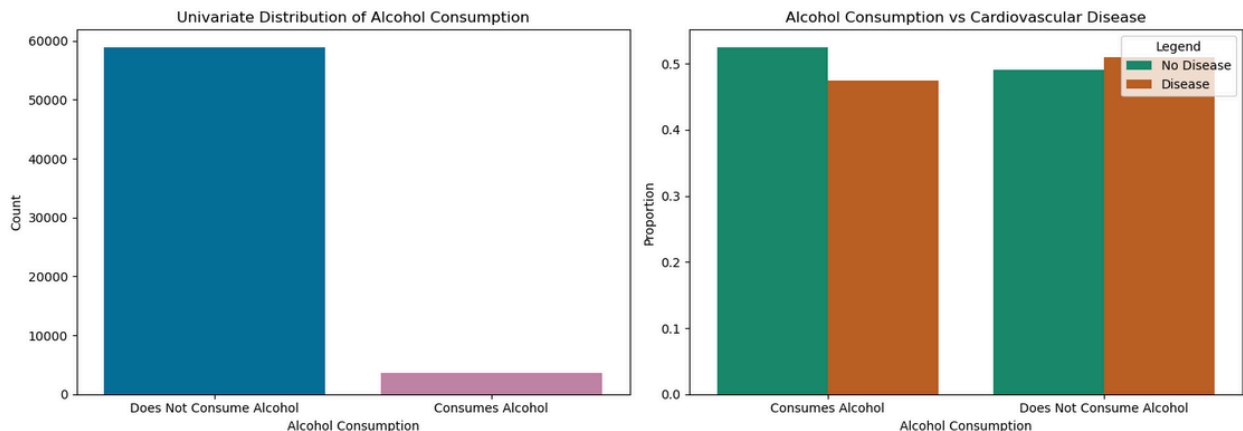
# Palettes
alco_palette = ['#0072B2', '#CC79A7']
cardio_palette = {'No Disease': '#009E73', 'Disease': '#D55E00'}

# --- Left: Univariate Alcohol ---
sns.countplot(x='alco_label', data=tmp, ax=axes[0], palette=alco_palette)
axes[0].set_title("Univariate Distribution of Alcohol Consumption")
axes[0].set_xlabel("Alcohol Consumption")
axes[0].set_ylabel("Count")

# --- Build proportion table for bivariate ---
prop = (pd.crosstab(tmp['alco_label'], tmp['cardio_label'], normalize='index')
        .stack().rename('proportion').reset_index())

# --- Right: Bivariate (Alcohol vs Cardio) ---
sns.barplot(x='alco_label', y='proportion', hue='cardio_label',
            data=prop, ax=axes[1], palette=cardio_palette,
            hue_order=['No Disease', 'Disease'])
axes[1].set_title("Alcohol Consumption vs Cardiovascular Disease")
axes[1].set_xlabel("Alcohol Consumption")
axes[1].set_ylabel("Proportion")
axes[1].legend(title="Legend")

plt.tight_layout()
plt.show()
```



The “active” variable shows that most patients report being physically active. Among active patients, the proportion with CVD is nearly evenly split 50/50, while among inactive patients, 54% have the disease. This highlights the role that physical activity potentially poses when it comes to the disease.

```
[30]: # Visualization for active

# Map labels for readability
tmp = df.assign(
    active_label = df['active'].map({0: 'Not Active', 1: 'Active'}),
    cardio_label = df['cardio'].map({0: 'No Disease', 1: 'Disease'})
)

# Proportion table for bivariate
prop = pd.crosstab(tmp['active_label'], tmp['cardio_label'], normalize='index')

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

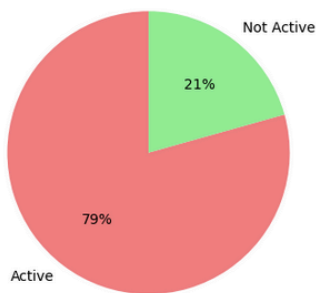
# --- Left: Univariate Active ---
active_counts = tmp['active_label'].value_counts()
axes[0].pie(active_counts, labels=active_counts.index, autopct='%1.0f%%',
            colors=['lightcoral', 'lightgreen'], startangle=90)
axes[0].set_title("Univariate Distribution of Physical Activity")

# --- Right: Bivariate (Active vs Cardio) ---
prop.plot(kind='bar', stacked=True, ax=axes[1], color=['lightblue', 'orange'])
axes[1].set_ylabel('Proportion')
axes[1].set_xlabel('Physical Activity')
axes[1].set_title('Physical Activity vs Cardiovascular Disease')
axes[1].legend(title='Legend')

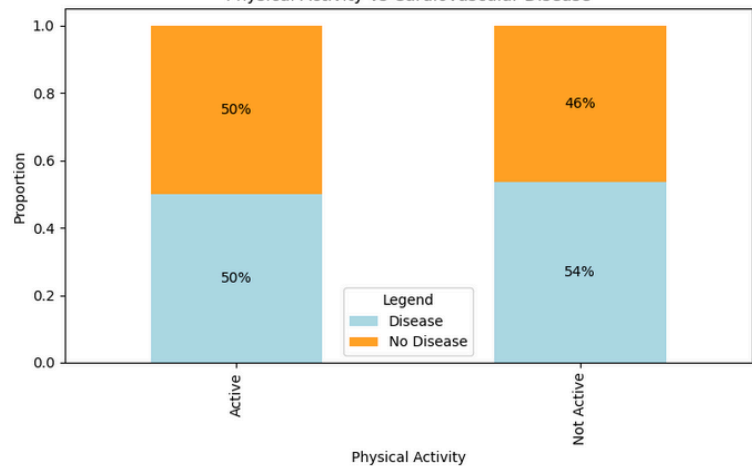
# Add percentage labels inside bars
for p in axes[1].patches:
    width, height = p.get_width(), p.get_height()
    if height > 0.03: # only annotate if large enough
        x, y = p.get_x() + width/2, p.get_y() + height/2
        axes[1].text(x, y, f"{height*100:.0f}%", ha='center', va='center', fontsize=10)

plt.tight_layout()
plt.show()
```

Univariate Distribution of Physical Activity



Physical Activity vs Cardiovascular Disease

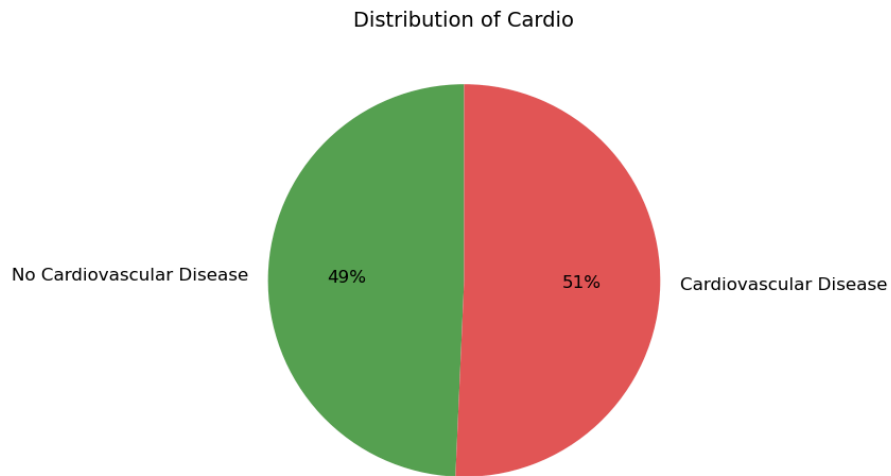


The target variable “cardio” shows an almost even split, with 51% of patients having the disease and 49% not having the disease. This balance supports the appropriateness of logistic regression, because the model is less likely to be biased by severe class imbalance.

```
[31]: # Distribution of cardio (target variable)

# Count values and map labels
cardio_counts = df['cardio'].value_counts().sort_index()
labels = ['No Cardiovascular Disease', 'Cardiovascular Disease']
colors = ['#54A24B', '#E45756'] # green for healthy, red for disease

plt.figure(figsize=(6,6))
plt.pie(cardio_counts, labels=labels, autopct='%1.0f%%', startangle=90,
        colors=colors, textprops={'fontsize': 12})
plt.title("Distribution of Cardio", fontsize=14)
plt.show()
```



Examination of the “gender” variable shows that there are more female patients than male patients overall. In both groups, the majority of patients had CVD; however, there is a small gap between those who have the disease and those who do not. This indicates that CVD affects both genders, but gender alone may not explain variation without considering other predictors.

```
[32]: # Visualization for gender

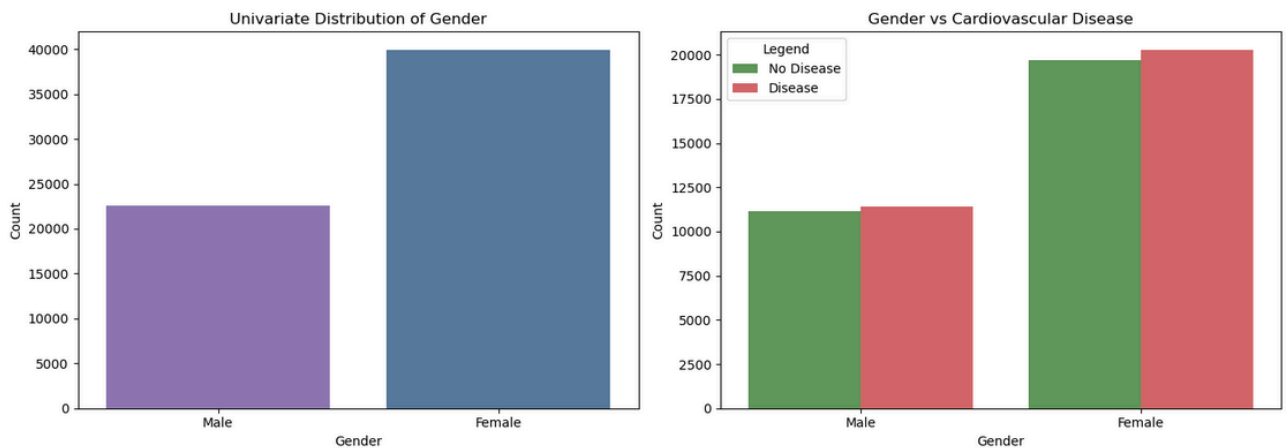
# Make labels
tmp = df.assign(
    gender_label = df['gender_male'].map({0: 'Female', 1: 'Male'}),
    cardio_label = df['cardio'].map({0: 'No Disease', 1: 'Disease'})
)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Left: Univariate Gender ---
sns.countplot(x='gender_label', data=tmp, ax=axes[0], palette=['#8E6BBE', '#4C78A8'])
axes[0].set_title("Univariate Distribution of Gender")
axes[0].set_xlabel("Gender")
axes[0].set_ylabel("Count")

# --- Right: Bivariate (Gender vs Cardio) ---
sns.countplot(x='gender_label', hue='cardio_label', data=tmp,
              ax=axes[1], palette={'No Disease': '#54A24B', 'Disease': '#E45756'})
axes[1].set_title("Gender vs Cardiovascular Disease")
axes[1].set_xlabel("Gender")
axes[1].set_ylabel("Count")
axes[1].legend(title="Legend")

plt.tight_layout()
plt.show()
```



The “cholesterol” variable shows that most patients had normal cholesterol levels. However, bivariate plots display that patients with above normal or well above normal cholesterol are disproportionately more likely to have CVD compared to those with normal levels.

```
[33]: # Visualization for cholesterol

# Recombine cholesterol dummy variables into one categorical variable
def chol_category(row):
    if row['cholesterol_above_normal'] == 1:
        return 'Above Normal'
    elif row['cholesterol_well_above_normal'] == 1:
        return 'Well Above Normal'
    else:
        return 'Normal'

tmp = df.copy()
tmp['cholesterol_label'] = tmp.apply(chol_category, axis=1)
tmp['cardio_label'] = tmp['cardio'].map({0: 'No Disease', 1: 'Disease'})

# Proportion table for bivariate
prop = (pd.crosstab(tmp['cholesterol_label'], tmp['cardio_label'], normalize='index')
        .stack().rename('proportion').reset_index())

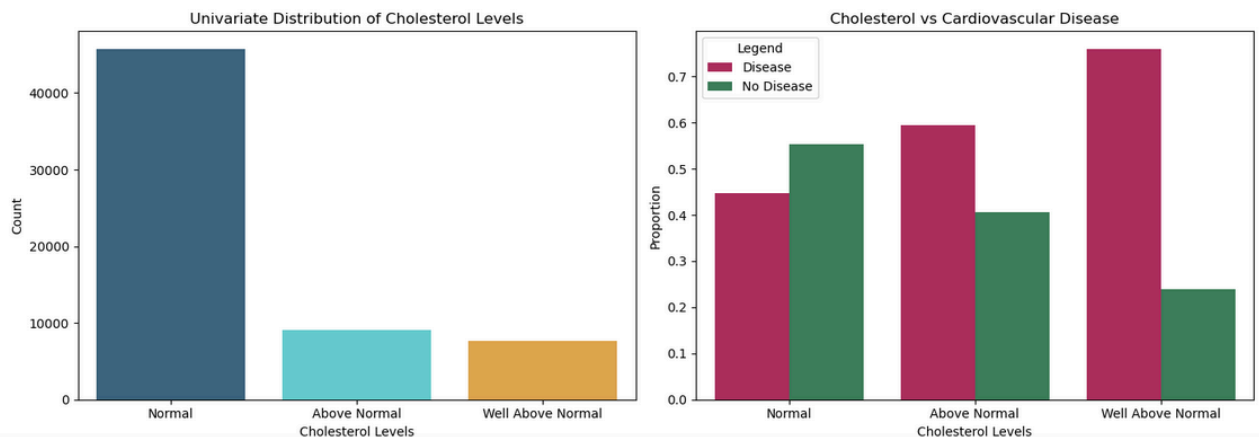
# New palettes
chol_palette = ['#33658A', '#55DDE0', '#F6AE2D'] # Normal, Above, Well Above
cardio_palette = {'No Disease': '#2E8B57', 'Disease': '#C2185B'}

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Left: Univariate Cholesterol ---
sns.countplot(x='cholesterol_label', data=tmp,
              order=['Normal', 'Above Normal', 'Well Above Normal'],
              palette=chol_palette, ax=axes[0])
axes[0].set_title("Univariate Distribution of Cholesterol Levels")
axes[0].set_xlabel("Cholesterol Levels")
axes[0].set_ylabel("Count")

# --- Right: Bivariate (Cholesterol vs Cardio) ---
sns.barplot(x='cholesterol_label', y='proportion', hue='cardio_label',
            data=prop, order=['Normal', 'Above Normal', 'Well Above Normal'],
            palette=cardio_palette, ax=axes[1])
axes[1].set_title("Cholesterol vs Cardiovascular Disease")
axes[1].set_xlabel("Cholesterol Levels")
axes[1].set_ylabel("Proportion")
axes[1].legend(title="Legend")

plt.tight_layout()
plt.show()
```



Similarly, for the “gluc” variable, most patients have normal levels, but the likelihood of the disease is higher among those with above normal and well above normal glucose levels. These results show that high cholesterol and impaired glucose tolerance could be major risk factors for the disease.

```
[34]: # Visualization for glucose

# Recombine glucose dummy variables into one categorical variable
def gluc_category(row):
    if row['glucose_above_normal'] == 1:
        return 'Above Normal'
    elif row['glucose_well_above_normal'] == 1:
        return 'Well Above Normal'
    else:
        return 'Normal'

tmp = df.copy()
tmp['glucose_label'] = tmp.apply(gluc_category, axis=1)
tmp['cardio_label'] = tmp['cardio'].map({0: 'No Disease', 1: 'Disease'})

# Make proportion table for bivariate
prop = (pd.crosstab(tmp['glucose_label'], tmp['cardio_label'], normalize='index')
        .stack().rename('proportion').reset_index())

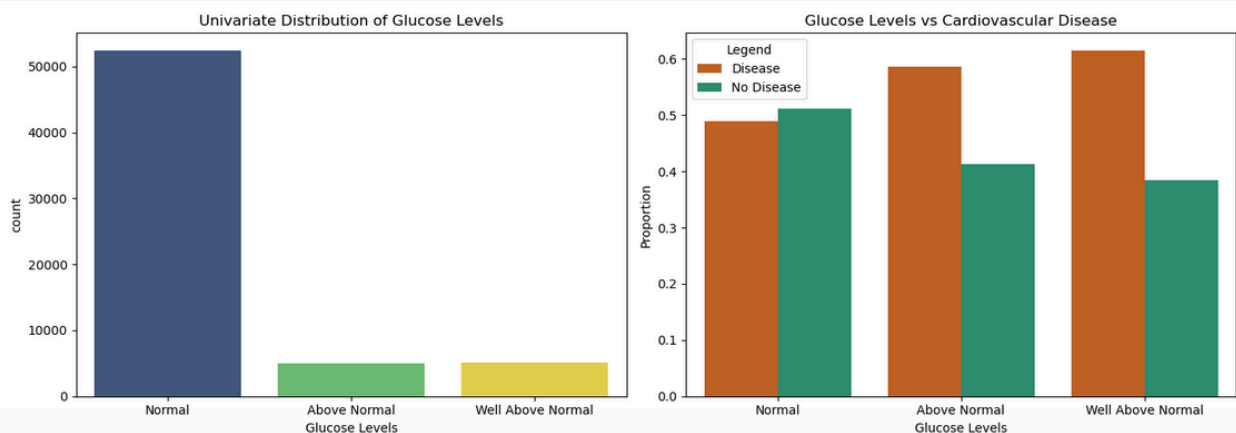
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# New palettes
gluc_palette = ['#3B528B', '#5DC863', '#FDE725']
cardio_palette = {'No Disease': '#1B9E77', 'Disease': '#D95F02'}

# --- Left: Univariate Glucose ---
sns.countplot(x='glucose_label', data=tmp,
              order=['Normal', 'Above Normal', 'Well Above Normal'],
              palette=gluc_palette, ax=axes[0])
axes[0].set_title("Univariate Distribution of Glucose Levels")
axes[0].set_xlabel("Glucose Levels")

# --- Right: Bivariate (Glucose vs Cardio) ---
sns.barplot(x='glucose_label', y='proportion', hue='cardio_label',
            data=prop, order=['Normal', 'Above Normal', 'Well Above Normal'],
            palette=cardio_palette, ax=axes[1])
axes[1].set_title("Glucose Levels vs Cardiovascular Disease")
axes[1].set_xlabel("Glucose Levels")
axes[1].set_ylabel("Proportion")
axes[1].legend(title='Legend')

plt.tight_layout()
plt.show()
```



Analysis of “bp_category” reveals that the majority of patients fall within hypertension stage 1. Bivariate plots show that patients in hypertension stage 2 had the highest proportion of CVD, while patients with normal, elevated, and hypertension stage 1 are more likely to not have the disease. This highlights the role hypertension severity levels have in determining cardiovascular outcomes.

```
[35]: # Visualization for bp_category

# Recombine BP dummies into one categorical variable
def bp_category(row):
    if row['bp_elevated'] == 1:
        return 'Elevated'
    elif row['bp_hypertension_stage_1'] == 1:
        return 'Hypertension Stage 1'
    elif row['bp_hypertension_stage_2'] == 1:
        return 'Hypertension Stage 2'
    else:
        return 'Normal'

tmp = df.copy()
tmp['bp_label'] = tmp.apply(bp_category, axis=1)
tmp['cardio_label'] = tmp['cardio'].map({0: 'No Disease', 1: 'Disease'})

# Proportion table for bivariate
prop = (pd.crosstab(tmp['bp_label'], tmp['cardio_label'], normalize='index')
        .stack().rename('proportion').reset_index())

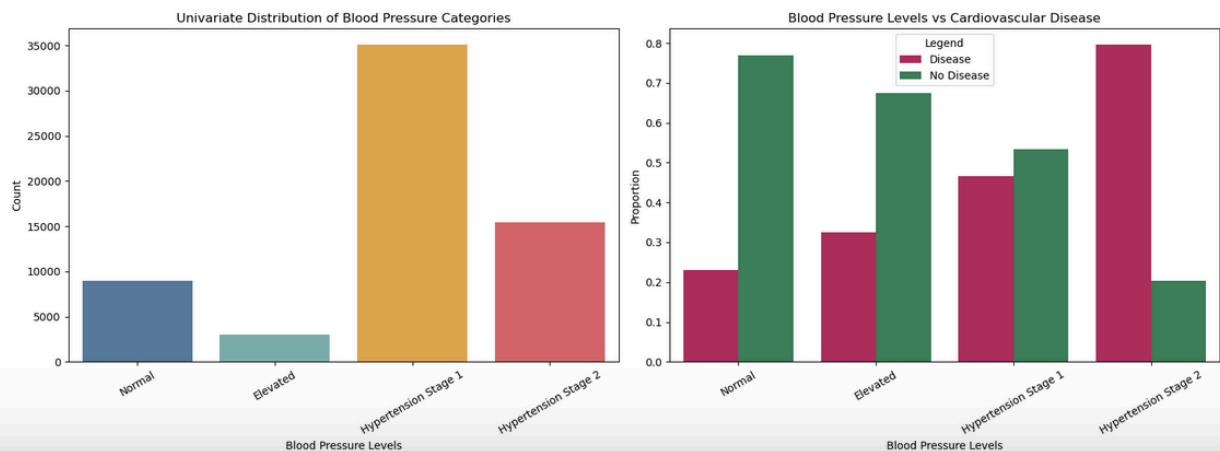
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# New palette
bp_palette = ['#4C78A8', '#72B7B2', '#F6AE2D', '#E45756', '#B279A2']
cardio_palette = {'No Disease': '#2E8B57', 'Disease': '#C2185B'}

# --- Left: Univariate Blood Pressure ---
sns.countplot(x='bp_label', data=tmp,
              order=['Normal', 'Elevated', 'Hypertension Stage 1', 'Hypertension Stage 2'],
              palette=bp_palette, ax=axes[0])
axes[0].set_title("Univariate Distribution of Blood Pressure Categories")
axes[0].set_xlabel("Blood Pressure Levels")
axes[0].set_ylabel("Count")
axes[0].tick_params(axis='x', rotation=30)

# --- Right: Bivariate (Blood Pressure vs Cardiovascular Disease) ---
sns.barplot(x='bp_label', y='proportion', hue='cardio_label',
            data=prop,
            order=['Normal', 'Elevated', 'Hypertension Stage 1', 'Hypertension Stage 2'],
            palette=cardio_palette, ax=axes[1])
axes[1].set_title("Blood Pressure Levels vs Cardiovascular Disease")
axes[1].set_xlabel("Blood Pressure Levels")
axes[1].set_ylabel("Proportion")
axes[1].legend(title="Legend")
axes[1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()
```



The cleaned dataset is saved, using the following code:

```
[36]: # Save cleaned dataset
      df.to_csv('cardio_logistic.csv')
```

D. Analysis

The data is analyzed using logistic regression, a statistical technique appropriate for predicting a binary outcome (GeeksforGeeks, 2025). The target variable “cardio” is binary, coded as 0 for patients without cardiovascular disease and 1 for patients with the disease. To ensure unbiased model evaluation, the dataset is split into training and testing subsets using the `train_test_split` function from `scikit-learn`. The test size is set to 20%, the random state is set to 42 for reproducibility, and stratification is applied to the dependent variable to maintain class balance between training and test sets (GeeksforGeeks, 2025). Predictor variables (X) include demographic, lifestyle, and clinical factors, while the outcome variable (y) is the presence of cardiovascular disease.

```
[37]: from sklearn.model_selection import train_test_split

      X = df[['age', 'bmi', 'smoke', 'alco', 'active', 'gender_male', 'cholesterol_above_normal',
              'cholesterol_well_above_normal', 'glucose_above_normal', 'glucose_well_above_normal', 'bp_elevated',
              'bp_hypertension_stage_1', 'bp_hypertension_stage_2']]
      y = df['cardio']

      # Split data into training and testing sets
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```


Next, the logistic regression model is fitted and the initial model is created. To refine the model, backward elimination is performed based on two criteria: Variance Inflation Factor (VIF), used to detect multicollinearity, and statistical significance, measured by p-values. A threshold of VIF > 10 is used to indicate severe multicollinearity, (Singh, 2024) while predictors with p-values above 0.05 are considered statistically insignificant and candidates for removal.

```
[38]: # Create initial logistic regression model

X = df[['age', 'bmi', 'smoke', 'alco', 'active', 'gender_male', 'cholesterol_above_normal',
        'cholesterol_well_above_normal', 'glucose_above_normal', 'glucose_well_above_normal', 'bp_elevated',
        'bp_hypertension_stage_1', 'bp_hypertension_stage_2']]
y = df['cardio']

from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm

# Calculate VIFs for each independent variable
vif_data = pd.DataFrame({
    'Variable': X.columns,
    'VIF': [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})

# Display VIF
print(vif_data)

X = sm.add_constant(X) # Add intercept
initial_model = sm.Logit(y,X).fit()
print(initial_model.summary())
```

```

      Variable      VIF
0          age  27.207203
1          bmi  24.135531
2         smoke   1.374652
3          alco   1.203984
4         active   4.625421
5   gender_male   1.799614
6 cholesterol_above_normal  1.312184
7 cholesterol_well_above_normal  1.544910
8   glucose_above_normal   1.188494
9  glucose_well_above_normal   1.420007
10         bp_elevated   1.339654
11 bp_hypertension_stage_1   5.121771
12 bp_hypertension_stage_2   2.988324
Optimization terminated successfully.
      Current function value: 0.586037
      Iterations 6

Logit Regression Results
=====
Dep. Variable:          cardio      No. Observations:          62553
Model:              Logit      Df Residuals:          62539
Method:              MLE      Df Model:              13
Date:                Tue, 09 Sep 2025      Pseudo R-squ.:          0.1544
Time:                03:51:44      Log-Likelihood:          -36658.
Converged:              True      LL-Null:          -43352.
Covariance Type:      nonrobust      LLR p-value:          0.000
=====
              coef      std err      z      P>|z|      [0.025      0.975]
-----
const              -4.9141      0.090     -54.658      0.000      -5.090      -4.738
age                 0.0538      0.001     39.593      0.000      0.051      0.056
bmi                 0.0391      0.002     21.289      0.000      0.035      0.043
smoke              -0.1272      0.034     -3.739      0.000     -0.194     -0.061
alco               -0.2088      0.041     -5.070      0.000     -0.290     -0.128
active             -0.1891      0.022     -8.606      0.000     -0.232     -0.146
gender_male        0.0589      0.020      2.949      0.003      0.020      0.098
cholesterol_above_normal  0.4027      0.027     15.086      0.000      0.350      0.455
cholesterol_well_above_normal  1.1524      0.035     32.773      0.000      1.083      1.221
glucose_above_normal  0.0262      0.035      0.748      0.454     -0.042      0.095
glucose_well_above_normal -0.3594      0.039     -9.233      0.000     -0.436     -0.283
bp_elevated        0.3654      0.048      7.667      0.000      0.272      0.459
bp_hypertension_stage_1  0.8371      0.028     29.441      0.000      0.781      0.893
bp_hypertension_stage_2  2.2252      0.033     66.522      0.000      2.160      2.291
=====
```

In the initial model, the predictor variable “age” has the highest VIF at 27.21, suggesting redundancy with other variables. Therefore, “age” is removed and a second iteration is done.

In the second iteration, no predictors have VIF values exceeding 10, however the variable “glucose_above_normal” has a p-value of 0.227, greater than the 0.05 threshold, and is excluded in the third iteration.

```
*[39]: # Remove "age" with VIF of 27.207203
X = df[['bmi', 'smoke', 'alco', 'active', 'gender_male', 'cholesterol_above_normal',
        'cholesterol_well_above_normal', 'glucose_above_normal', 'glucose_well_above_normal', 'bp_elevated',
        'bp_hypertension_stage_1', 'bp_hypertension_stage_2']]
y = df['cardio']

# Calculate VIFs for each independent variable
vif_data = pd.DataFrame({
    'Variable': X.columns,
    'VIF': [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})

# Display VIF
print(vif_data)

X = sm.add_constant(X) # Add intercept
model = sm.Logit(y,X).fit()
print(model.summary())
```

	Variable	VIF
0	bmi	9.603249
1	smoke	1.374376
2	alco	1.203098
3	active	4.346221
4	gender_male	1.764513
5	cholesterol_above_normal	1.310768
6	cholesterol_well_above_normal	1.543880
7	glucose_above_normal	1.188121
8	glucose_well_above_normal	1.419244
9	bp_elevated	1.294681
10	bp_hypertension_stage_1	4.709073
11	bp_hypertension_stage_2	2.861677

Optimization terminated successfully.
Current function value: 0.598934
Iterations 5

Logit Regression Results

Dep. Variable:		cardio	No. Observations:	62553
Model:	Logit	Df Residuals:	62540	
Method:	MLE	Df Model:	12	
Date:	Tue, 09 Sep 2025	Pseudo R-squ.:	0.1358	
Time:	03:51:46	Log-Likelihood:	-37465.	
converged:	True	LL-Null:	-43352.	
Covariance Type:	nonrobust	LLR p-value:	0.000	

	coef	std err	z	P> z	[0.025	0.975]
const	-2.2450	0.057	-39.446	0.000	-2.357	-2.133
bmi	0.0416	0.002	22.945	0.000	0.038	0.045
smoke	-0.1731	0.034	-5.156	0.000	-0.239	-0.107
alco	-0.2368	0.041	-5.826	0.000	-0.317	-0.157
active	-0.1934	0.022	-8.926	0.000	-0.236	-0.151
gender_male	0.0506	0.020	2.567	0.010	0.012	0.089
cholesterol_above_normal	0.4227	0.026	16.033	0.000	0.371	0.474
cholesterol_well_above_normal	1.2676	0.035	36.548	0.000	1.200	1.336
glucose_above_normal	0.0417	0.035	1.209	0.227	-0.026	0.109
glucose_well_above_normal	-0.3135	0.039	-8.129	0.000	-0.389	-0.238
bp_elevated	0.4599	0.047	9.791	0.000	0.368	0.552
bp_hypertension_stage_1	0.9469	0.028	33.860	0.000	0.892	1.002
bp_hypertension_stage_2	2.3524	0.033	71.287	0.000	2.288	2.417

After “glucose_above_normal” is removed in the third iteration, all remaining predictors are determined to be statistically significant and free of multicollinearity, with p-values below 0.05 and VIF value less than 10.

```
*[40]: # Remove "glucose_above_normal" with p-value of 0.227
X = df[['bmi', 'smoke', 'alco', 'active', 'gender_male', 'cholesterol_above_normal',
        'cholesterol_well_above_normal', 'glucose_well_above_normal', 'bp_elevated',
        'bp_hypertension_stage_1', 'bp_hypertension_stage_2']]
y = df['cardio']

# Calculate VIFs for each independent variable
vif_data = pd.DataFrame({
    'Variable': X.columns,
    'VIF': [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})

# Display VIF
print(vif_data)

X = sm.add_constant(X) # Add intercept
final_model = sm.Logit(y,X).fit()
# Print final model
print(final_model.summary())
```

```

Variable      VIF
0          bmi  9.53774
1          smoke 1.374373
2           alco 1.202891
3          active 4.344685
4    gender_male 1.764428
5 cholesterol_above_normal 1.223364
6 cholesterol_well_above_normal 1.539952
7  glucose_well_above_normal 1.406560
8      bp_elevated 1.294251
9  bp_hypertension_stage_1 4.708710
10 bp_hypertension_stage_2 2.861509
Optimization terminated successfully.
Current function value: 0.598946
Iterations 5
```

```

Logit Regression Results
=====
Dep. Variable:      cardio    No. Observations:      62553
Model:              Logit    Df Residuals:           62541
Method:              MLE     Df Model:              11
Date:                Tue, 09 Sep 2025    Pseudo R-squ.:      0.1358
Time:                03:51:47    Log-Likelihood:     -37466.
Converged:           True      LL-Null:           -43352.
Covariance Type:     nonrobust    LLR p-value:        0.000
=====
```

	coef	std err	z	P> z	[0.025	0.975]
const	-2.2471	0.057	-39.500	0.000	-2.359	-2.136
bmi	0.0418	0.002	23.086	0.000	0.038	0.045
smoke	-0.1731	0.034	-5.155	0.000	-0.239	-0.107
alco	-0.2362	0.041	-5.812	0.000	-0.316	-0.157
active	-0.1935	0.022	-8.932	0.000	-0.236	-0.151
gender_male	0.0506	0.020	2.567	0.010	0.012	0.089
cholesterol_above_normal	0.4311	0.025	16.955	0.000	0.381	0.481
cholesterol_well_above_normal	1.2693	0.035	36.628	0.000	1.201	1.337
glucose_well_above_normal	-0.3175	0.038	-8.261	0.000	-0.393	-0.242
bp_elevated	0.4593	0.047	9.778	0.000	0.367	0.551
bp_hypertension_stage_1	0.9470	0.028	33.861	0.000	0.892	1.002
bp_hypertension_stage_2	2.3528	0.033	71.302	0.000	2.288	2.417

The final model retained the following variables:

- bmi
- smoke
- alco
- active
- gender_male
- cholesterol_above_normal

- cholesterol_well_above_normal
- glucose_well_above_normal
- bp_elevated
- bp_hypertension_stage_1
- bp_hypertension_stage_2

Logistic regression is chosen as the analysis technique because it is specifically designed to model the probability of a binary outcome as a function of multiple predictor variables. It provides interpretable coefficients that can be transformed into odds ratios, making it possible to quantify how each predictor increases or decreases the likelihood of cardiovascular disease. (Wikipedia, n.d.) This is important in healthcare research, where understanding the direction and strength of relationships between risk factors and a disease can inform clinical decision-making and public health strategies. Another advantage of logistic regression is the ability to incorporate both continuous and categorical predictors in the same model.

A limitation of logistic regression is its assumption of linearity of the logit, meaning that the relationship between predictors and the log-odds of the outcome must be linear. Violations to this assumption can greatly impact model accuracy. The effects of these violations are that the model might overestimate or underestimate the effect of a predictor, predicted probabilities could be too high or too low across predictor variables, and the model's ability to accurately classify cases into 0 (no disease) or 1 (disease) decreases.

E. Data Summary and Implications

The final logistic regression model produces a pseudo R^2 value of 0.1358, indicating that 13.58% of the variance in cardiovascular disease outcomes are explained by the predictors included in the model. This value suggests that other unmeasured factors also influence CVD, it still demonstrates that demographic, lifestyle, and clinical variables can contribute meaningfully to disease risk.

To assess predictive power, the model is evaluated using a confusion matrix and accuracy score. The confusion matrix is made up of four values: true positives, true negatives, false positives, false negatives. The confusion matrix reveals that the model correctly predicts 4,675 patients without cardiovascular disease (true negatives) and 4,124 patients with the disease (true positives). However, it misclassifies 1,492 patients as not having CVD when they actually do (false negatives) and 2,220 patients as having the disease when they do not (false positives). These results produce an overall accuracy score of 70%.

```
[41]: from sklearn.linear_model import LogisticRegression
      from sklearn.metrics import confusion_matrix

      # Create confusion matrix and accuracy

      # Initialize and train the logistic regression model
      logreg = LogisticRegression()
      logreg.fit(X_train, y_train)

      # Predict the target variable for the test set
      y_pred = logreg.predict(X_test)

      # Print accuracy on the test set
      print('Accuracy of logistic regression classifier on test set: {:.2f}'.format(logreg.score(X_test, y_test)))

      # Compute the confusion matrix
      final_matrix = confusion_matrix(y_test, y_pred)

      # Print the confusion matrix
      print("Confusion Matrix:")
      print(final_matrix)

      Accuracy of logistic regression classifier on test set: 0.70
      Confusion Matrix:
      [[4675 1492]
       [2220 4124]]
```

The accuracy score suggests that the model has moderate predictive power, but the high number of false negatives has clinical implications. Missing patients who truly have cardiovascular disease may delay treatment and increase health risks. On the other hand, false positives may lead to unnecessary testing, which can be an inconvenience but much minor compared to missed diagnoses. The results of the confusion matrix and the accuracy score shows that the model can be used to identify general risk patterns, but should not be the sole source for clinical decision-making.

Cook's distance is calculated to assess whether any observations have an unusually strong influence on the logistic regression model. A common threshold for finding influential observations is values greater than 0.02, though others suggest $4/n$ (with n equal to the number of observations) as a cutoff. In the model, the maximum Cook's distance is at ~ 0.0030 , well below the threshold. As shown in the plot, no data points exceeded the cutoff, indicating that no single observation disproportionately influences the model results.

```
[42]: # Cook's distance to check for outliers in final model
# Code adapted from Statology:
# Bobbit, Z. (2020). How to Calculate Cook's Distance in Python
# Retrieved September 7 2025, from https://www.statology.org/cooks-distance-python/

# Suppress scientific notation
np.set_printoptions(suppress=True)

# Create instance of influence
influence = final_model.get_influence()

# Obtain Cook's distance for each observation
cooks = influence.cooks_distance

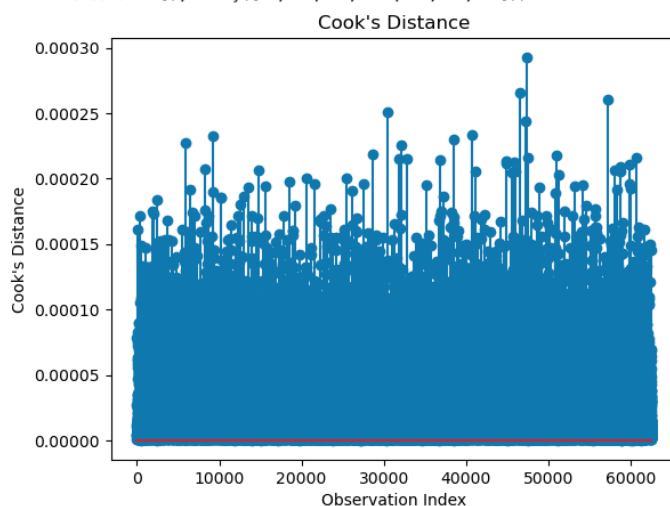
# Display Cook's distances
print(cooks)

# Visualize cook's distance

plt.stem(np.arange(len(cooks[0])), cooks[0], markerfmt="o")
plt.title("Cook's Distance")
plt.xlabel("Observation Index")
plt.ylabel("Cook's Distance")
plt.show()

# Identify influential observations
influential_points = np.where(cooks[0] > 0.02)
print("Indices of influential observations:", influential_points[0])

(array([0.00000472, 0.0000006 , 0.00001041, ..., 0.00000166, 0.0000117 ,
        0.00001441]), array([1., 1., 1., ..., 1., 1., 1.]))
```



Indices of influential observations: []

Analysis of odds ratio provides deeper insight into the predictors' influence on cardiovascular disease. Higher BMI increases the odds of cardiovascular disease by 4% per unit increase. Among lifestyle factors, smoking, alcohol consumption, and physical activity show odds ratios below 1, meaning that patients who smoke or consume alcohol are less likely to have

cardiovascular disease. Smokers have 15.89% lower odds of CVD, those who consume alcohol have 21.04% lower odds, and physically active patients have 17.60% lower odds. However, through medical research, it is known that smoking and heavy alcohol use does in fact increase risk of cardiovascular disease (Mukamal, 2006, 199-202). This mismatch in the analysis could be the result of confounding variables. For example, younger patients could be more likely to report smoking or alcohol use, and younger age lowers cardiovascular disease risk. Then the model erroneously adopts that association. Being male increases odds of cardiovascular disease by 5.19%. Clinical variables are particularly strong predictors. Patients with cholesterol levels well above normal have 3.56 times the odds of cardiovascular disease compared to those with normal levels, while those in hypertension stage 2 have over 10 times the odds. This suggests the critical role of blood pressure and cholesterol in cardiovascular disease risk.

```

•[43]: # Get coefficients and calculate odds ratios
coefficients = final_model.params
odds_ratios = np.exp(coefficients)

# Exclude intercept from analysis
odds_ratios = odds_ratios.drop('const', errors='ignore')

# Interpret each odds ratio
for predictor, or_value in odds_ratios.items():
    if or_value > 1:
        print(f"The odds ratio for {predictor} is {or_value:.2f}, meaning that as {predictor} increases by one unit, the odds of the outcome increase by {(or_value - 1) * 100:.2f}%.")
    elif or_value < 1:
        print(f"The odds ratio for {predictor} is {or_value:.2f}, meaning that as {predictor} increases by one unit, the odds of the outcome decrease by {(1 - or_value) * 100:.2f}%.")
    else:
        print(f"The odds ratio for {predictor} is {or_value:.2f}, meaning that {predictor} has no effect on the odds of the outcome.")

```

The odds ratio for bmi is 1.04, meaning that as bmi increases by one unit, the odds of the outcome increase by 4.27%.
 The odds ratio for smoke is 0.84, meaning that as smoke increases by one unit, the odds of the outcome decrease by 15.89%.
 The odds ratio for alco is 0.79, meaning that as alco increases by one unit, the odds of the outcome decrease by 21.04%.
 The odds ratio for active is 0.82, meaning that as active increases by one unit, the odds of the outcome decrease by 17.60%.
 The odds ratio for gender_male is 1.05, meaning that as gender_male increases by one unit, the odds of the outcome increase by 5.19%.
 The odds ratio for cholesterol_above_normal is 1.54, meaning that as cholesterol_above_normal increases by one unit, the odds of the outcome increase by 53.90%.
 The odds ratio for cholesterol_well_above_normal is 3.56, meaning that as cholesterol_well_above_normal increases by one unit, the odds of the outcome increase by 255.85%.
 The odds ratio for glucose_well_above_normal is 0.73, meaning that as glucose_well_above_normal increases by one unit, the odds of the outcome decrease by 27.20%.
 The odds ratio for bp_elevated is 1.58, meaning that as bp_elevated increases by one unit, the odds of the outcome increase by 58.29%.
 The odds ratio for bp_hypertension_stage_1 is 2.58, meaning that as bp_hypertension_stage_1 increases by one unit, the odds of the outcome increase by 157.79%.
 The odds ratio for bp_hypertension_stage_2 is 10.52, meaning that as bp_hypertension_stage_2 increases by one unit, the odds of the outcome increase by 951.51%.

A limitation of the data analysis is that the dataset explains only 13.58% of the variance in cardiovascular disease outcomes, leaving most of the variation unexplained. This indicates that factors such as genetic predisposition, dietary intake, stress, and medication use are not taken into account. Additionally, using self-reported lifestyle measures, especially for physical activity, may introduce bias, limiting the accuracy of predictors.

Within the context of the research question, the analysis results suggest that hypertension severity and cholesterol levels are the strongest predictors of cardiovascular disease. Based on these findings, a recommended course of action is to prioritize monitoring and early intervention for patients with stage 2 hypertension and significantly elevated cholesterol, since these groups

carry the most risk. Public health initiatives can be implemented, emphasizing weight management and promoting a healthier lifestyle.

Two directions for future study are recommended. First, future analyses should incorporate additional variables such as family history, genetic markers, or dietary information, which might explain a greater portion of variance in CVD outcomes. Second, a more advanced modeling technique such as random forest could be used to explore nonlinear relationships between variables, potentially improving predictive accuracy greatly compared to logistic regression.

F. Sources

Bobbit, Z. (2021, October 21). *A Guide to Multicollinearity & VIF in Regression*. Statology.

<https://www.statology.org/multicollinearity-regression/>

Bobbitt, Z. (2020, October 13). *The 6 Assumptions of Logistic Regression (With Examples)*.

Statology. Retrieved September 9, 2025, from

<https://www.statology.org/assumptions-of-logistic-regression/>

CDC. (2024, March 19). *Adult BMI Categories | BMI*. CDC. Retrieved September 9, 2025, from

<https://www.cdc.gov/bmi/adult-calculator/bmi-categories.html>

CDC. (2024, October 24). *Heart Disease Facts | Heart Disease*. CDC. Retrieved September 9,

2025, from <https://www.cdc.gov/heart-disease/data-research/facts-stats/index.html>

GeeksforGeeks. (2025, July 23). *Binary Logistic Regression*. GeeksforGeeks. Retrieved

September 9, 2025, from

<https://www.geeksforgeeks.org/maths/binary-logistic-regression/>

GeeksforGeeks. (2025, July 23). *Stratified Sampling in Machine Learning*. GeeksforGeeks.

Retrieved September 9, 2025, from

<https://www.geeksforgeeks.org/machine-learning/stratified-sampling-in-machine-learning/>

/

Mukamal, K. J. (2006). The effects of smoking and drinking on cardiovascular disease and risk factors. *Alcohol research & health : the journal of the National Institute on Alcohol Abuse and Alcoholism*, 29(3), 199–202.

<https://pmc.ncbi.nlm.nih.gov/articles/PMC6527044/>

Singh, V. (2024, November 18). *Variance Inflation Factor: How to Detect Multicollinearity*.

DataCamp. Retrieved September 9, 2025, from

<https://www.datacamp.com/tutorial/variance-inflation-factor>

Thevapalan, A. (2024, September 10). *Winsorized Mean: What You Need to Know to Handle*

Outliers. DataCamp. Retrieved September 9, 2025, from

<https://www.datacamp.com/tutorial/winsorized-mean>

Welkins, C. (n.d.). *Cardiovascular disease dataset*. Kaggle.

<https://www.kaggle.com/datasets/colewelkins/cardiovascular-disease>

Wikipedia. (n.d.). *Odds ratio - Wikipedia*. Wikipedia, the free encyclopedia. Retrieved

September 9, 2025, from https://en.wikipedia.org/wiki/Odds_ratio